Comparative Analysis of Netflix Stock Performance Using Multiple Machine Learning Models

Author: Ryota Theodora

Abstract:

This study investigates the stock performance of Netflix (NFLX) using five machine learning models: Decision Trees, Random Forests, K-Nearest Neighbors (KNN), Gradient Boosting, and Support Vector Machines (SVM). The models aim to predict stock price movements based on historical data obtained from Yahoo Finance. The study evaluates the models' ability to predict short-term stock price fluctuations and assesses the accuracy of predictions using standard evaluation metrics such as accuracy, precision, recall, and F1-score. The analysis provides insights into the effectiveness of these machine learning techniques in the domain of stock market prediction.

Keywords:

Netflix stock, machine learning, decision tree, random forest, k-nearest neighbors, gradient boosting, support vector machines, stock price prediction, yfinance, predictive modeling

Introduction

Summary of the Study and Data:

In this project, we use historical stock price data of Netflix (NFLX) obtained from Yahoo Finance via yfinance. The dataset includes features such as open, close, high, low prices, and volume. We will train five different machine learning models: Decision Trees, Random Forests, K-Nearest Neighbors (KNN), Gradient Boosting, and Support Vector Machines (SVM). The objective is to predict future stock price movements and compare the performance of these models.

Key Questions:

- Can Decision Trees, Random Forests, K-Nearest Neighbors (KNN), Gradient Boosting, and Support Vector Machines (SVM) effectively predict Netflix's stock price movements?
- How do these machine learning models compare in terms of prediction accuracy and generalization?
- Which features are most important for predicting stock price movement?

Body

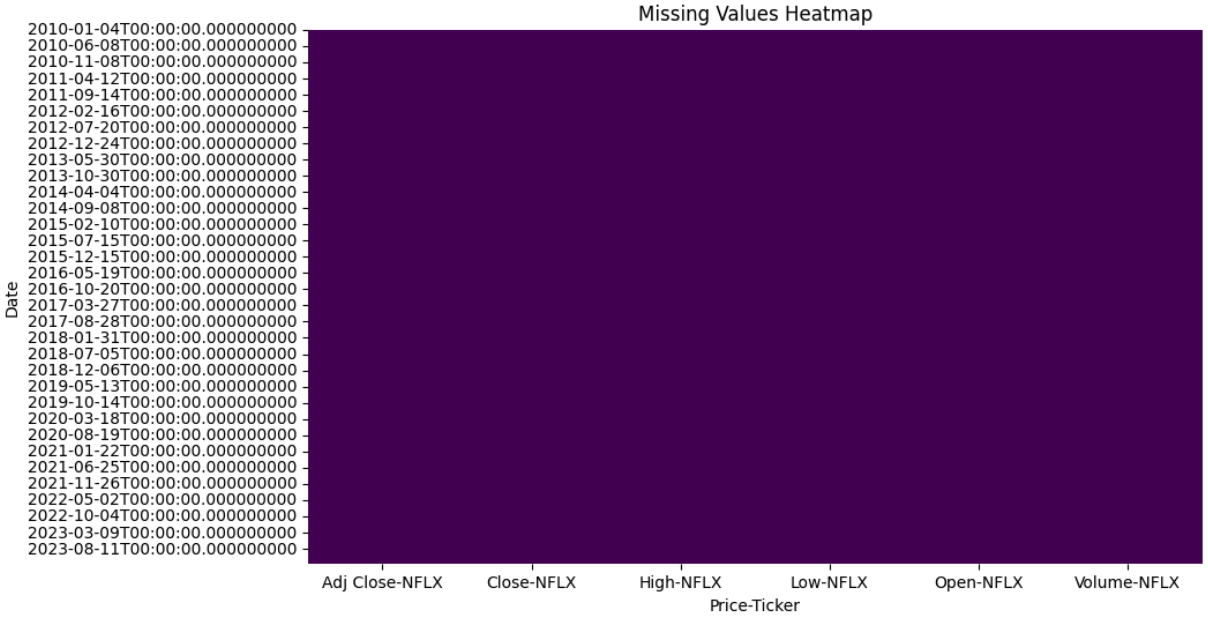
Data

To obtain the stock data for Netflix, we will use the yfinance library. The data will be used to build features and labels for our predictive models.

In [178…

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import zscore

# Download NFLX stock data from yfinance
stock_data = yf.download('NFLX', start='2010-01-01', end='2024-01-01')

# Check for missing values
print("Missing values in each column:")
print(stock_data.isna().sum())

# Visualize missing values (if any)
plt.figure(figsize=(10, 6))
sns.heatmap(stock_data.isna(), cbar=False, cmap='viridis', cbar_kws={'label': 'Miss
plt.title('Missing Values Heatmap')
plt.show()

# Z-scores for each column (excluding 'Date' and other non-numeric columns)
z_scores = zscore(stock_data[['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume'

# Identify outliers (Z-score > 3)
outliers = np.abs(z_scores) > 3

# Show how many outliers per column
outlier_count = outliers.sum(axis=0)
print("Number of outliers in each column (Z-score > 3):")
print(outlier_count)
```

```
[**********100%**********]  1 of 1 completed
Missing values in each column:
Price      Ticker
Adj Close  NFLX      0
Close      NFLX      0
High       NFLX      0
Low        NFLX      0
Open       NFLX      0
Volume     NFLX      0
dtype: int64
```




```
Number of outliers in each column (Z-score > 3):
Price      Ticker
Open       NFLX       0
High       NFLX       0
Low        NFLX       0
Close      NFLX       0
Adj Close  NFLX       0
Volume     NFLX      73
dtype: int64
```

No missing values were found in our data. The Volume may have outliers due to external factors, such as news, earning reports, or other significant events. In our case of short-term predictions, the volume here might not be relevant.

Feature Engineering:

We will create features like moving averages and daily percentage changes in the closing price to predict future stock movements.

In [179…

```
# Adding technical indicators (for example, moving averages)
stock_data['MA_50'] = stock_data['Close'].rolling(window=50).mean()
stock_data['MA_200'] = stock_data['Close'].rolling(window=200).mean()

# Adding daily percentage change in closing prices
stock_data['Pct_Change'] = stock_data['Close'].pct_change()

# Drop rows with NaN values that result from the moving averages and pct_change
stock_data = stock_data.dropna()

# Feature columns: Open, Close, High, Low, Volume, MA_50, MA_200, Pct_Change
features = ['Open', 'High', 'Low', 'Volume', 'MA_50', 'MA_200', 'Pct_Change']

# Labels: Stock price movement (binary: 1 if price goes up, 0 if it goes down)
stock_data['Price_Up'] = (stock_data['Close'].shift(-1) > stock_data['Close']).asty
```

Methods

Split Data for Training and Testing

Split the data into training and testing sets for model evaluation.

In [180…
```
from sklearn.model_selection import train_test_split

# Features and Labels
X = stock_data[features]
y = stock_data['Price_Up']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Machine Learning Models

Implement and evaluate the following models:

1. Decision Tree
2. Random Forests
3. K-Nearest Neighbors (KNN)
4. Gradient Boosting
5. Support Vector Machines (SVM)

1. Decision Tree

In [181…
```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
dt_predictions = dt_model.predict(X_test)
```

2. Random Forests

In [182…
```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_test)
```

3. K-Nearest Neighbors (KNN)

In [183…
```
from sklearn.neighbors import KNeighborsClassifier
knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
knn_predictions = knn_model.predict(X_test)
```

4. Gradient Boosting

In [184…
```
from sklearn.ensemble import GradientBoostingClassifier
gb_model = GradientBoostingClassifier(random_state=42)
```

```
gb_model.fit(X_train, y_train)
gb_predictions = gb_model.predict(X_test)
```

5. Support Vector Machines (SVM)

In [185…

```
from sklearn.svm import SVC
svm_model = SVC(kernel='rbf', random_state=42)
svm_model.fit(X_train, y_train)
svm_predictions = svm_model.predict(X_test)
```

Model Evaluation

Evaluate each model using classification metrics such as **accuracy**, **precision**, **recall**, and **F1-Score**

In [186…

```
from sklearn.metrics import classification_report, accuracy_score

# Evaluation
print("Decision Tree Classification Report:")
print(classification_report(y_test, dt_predictions))

print("Random Forest Classification Report:")
print(classification_report(y_test, rf_predictions))

print("K-Nearest Neighbors Classification Report:")
print(classification_report(y_test, knn_predictions))

print("Gradient Boosting Classification Report:")
print(classification_report(y_test, gb_predictions))

print("Support Vector Machines Classification Report:")
print(classification_report(y_test, svm_predictions))

# Accuracy Scores
print(f"Decision Tree Accuracy: {accuracy_score(y_test, dt_predictions):.4f}")
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf_predictions):.4f}")
print(f"K-Nearest Neighbors Accuracy: {accuracy_score(y_test, knn_predictions):.4f}
print(f"Gradient Boosting Accuracy: {accuracy_score(y_test, gb_predictions):.4f}")
print(f"Support Vector Machines Accuracy: {accuracy_score(y_test, svm_predictions):
```

```
Decision Tree Classification Report:
              precision    recall  f1-score   support

           0       0.49      0.51      0.50       319
           1       0.53      0.50      0.51       346

    accuracy                           0.51       665
   macro avg       0.51      0.51      0.51       665
weighted avg       0.51      0.51      0.51       665

Random Forest Classification Report:
              precision    recall  f1-score   support

           0       0.51      0.55      0.53       319
           1       0.55      0.51      0.53       346

    accuracy                           0.53       665
   macro avg       0.53      0.53      0.53       665
weighted avg       0.53      0.53      0.53       665

K-Nearest Neighbors Classification Report:
              precision    recall  f1-score   support

           0       0.46      0.46      0.46       319
           1       0.50      0.50      0.50       346

    accuracy                           0.48       665
   macro avg       0.48      0.48      0.48       665
weighted avg       0.48      0.48      0.48       665

Gradient Boosting Classification Report:
              precision    recall  f1-score   support

           0       0.49      0.46      0.47       319
           1       0.52      0.55      0.54       346

    accuracy                           0.51       665
   macro avg       0.50      0.50      0.50       665
weighted avg       0.51      0.51      0.51       665

Support Vector Machines Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.29      0.37       319
           1       0.52      0.71      0.60       346

    accuracy                           0.51       665
   macro avg       0.50      0.50      0.48       665
weighted avg       0.50      0.51      0.49       665

Decision Tree Accuracy: 0.5068
Random Forest Accuracy: 0.5323
K-Nearest Neighbors Accuracy: 0.4812
Gradient Boosting Accuracy: 0.5068
Support Vector Machines Accuracy: 0.5083
```

Cross Validation

In [187…
```
from sklearn.model_selection import cross_val_score

# Cross-validation for Random Forest
rf_cv_scores = cross_val_score(rf_model, X, y, cv=5)
print(f"Random Forest Cross-validation Scores: {rf_cv_scores}")
print(f"Mean Cross-validation Score: {rf_cv_scores.mean()}")

# Cross-validation for SVM
svm_cv_scores = cross_val_score(svm_model, X, y, cv=5)
print(f"SVM Cross-validation Scores: {svm_cv_scores}")
print(f"Mean Cross-validation Score: {svm_cv_scores.mean()}")
```

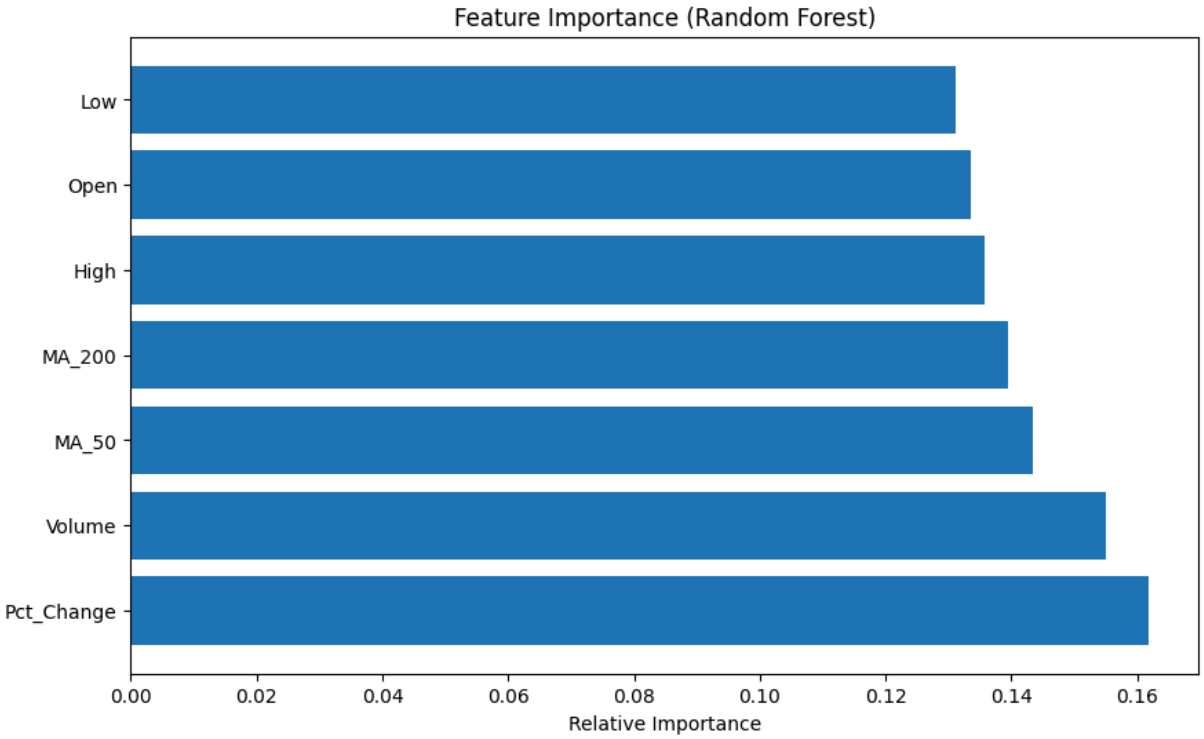
```
Random Forest Cross-validation Scores: [0.49924812 0.45413534 0.50526316 0.5376506
0.50451807]
Mean Cross-validation Score: 0.5001630582480298
SVM Cross-validation Scores: [0.50075188 0.50977444 0.51278195 0.50903614 0.5135542
2]
Mean Cross-validation Score: 0.509179726424495
```

Random Forests

I. Feature Importance

Feature importance helps identify which factors contribute the most to model predictions. Here's the visualization of feature importance from the Random Forest model:

In [188…
```
import matplotlib.pyplot as plt
import numpy as np

# Plot feature importance for Random Forest
rf_importances = rf_model.feature_importances_
rf_indices = np.argsort(rf_importances)[::-1]
plt.figure(figsize=(10, 6))
plt.title("Feature Importance (Random Forest)")
plt.barh(range(len(rf_indices)), rf_importances[rf_indices], align="center")
plt.yticks(range(len(rf_indices)), [features[i] for i in rf_indices])
plt.xlabel("Relative Importance")
plt.show()
```

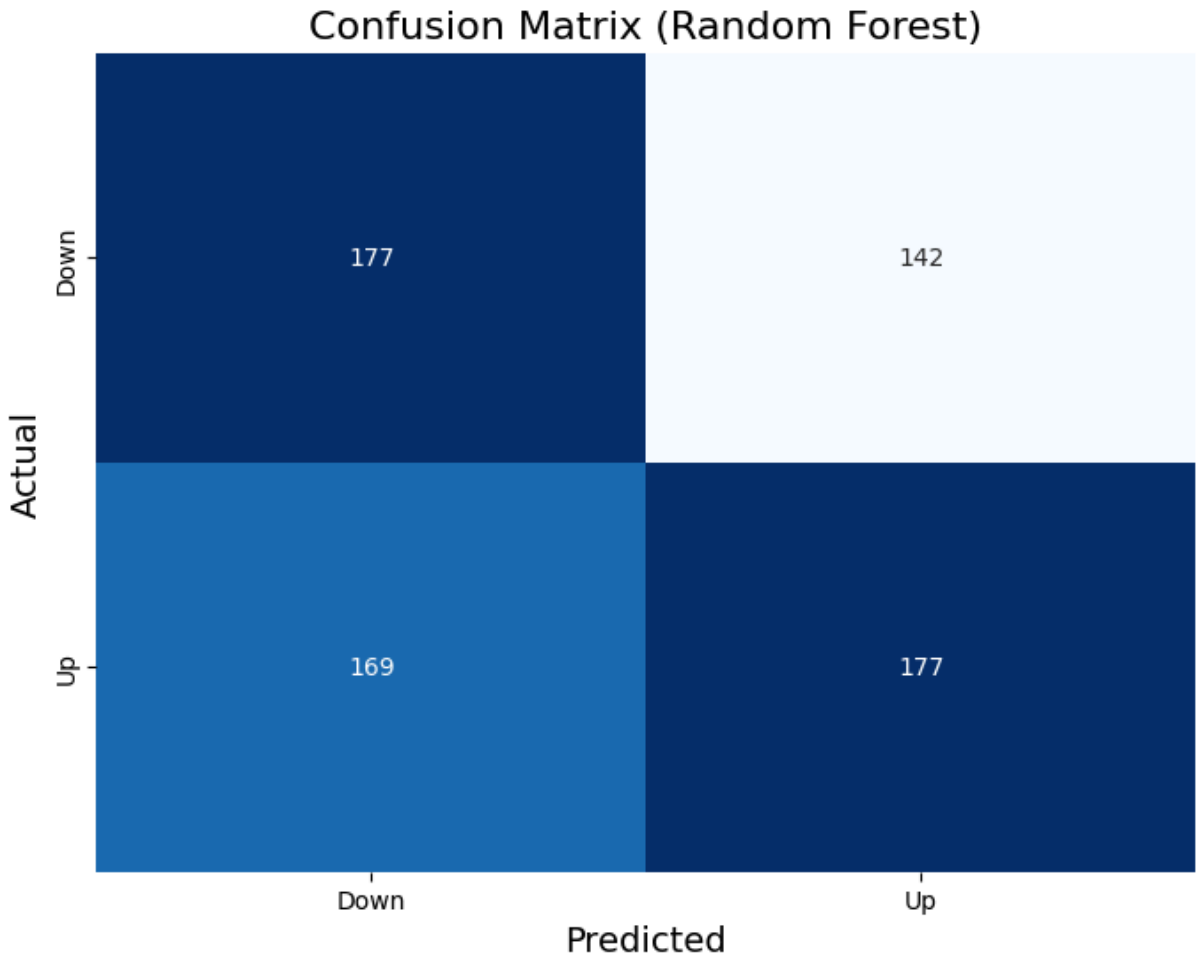


II. Confusion Matrix and Classification Report

Confusion matrix helps visualize the performance of the Random Forest classifier. Shown below:

In [189…

```
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# Random Forest Confusion Matrix
rf_cm = confusion_matrix(y_test, rf_predictions)

plt.figure(figsize=(8, 6))
sns.heatmap(rf_cm, annot=True, fmt='d', cmap='Blues', cbar=False, xticklabels=['Dow
plt.title('Confusion Matrix (Random Forest)', fontsize=16)
plt.xlabel('Predicted', fontsize=14)
plt.ylabel('Actual', fontsize=14)
plt.show()

# Random Forest Classification Report
print("Random Forest Classification Report:")
print(classification_report(y_test, rf_predictions))
```




```
Random Forest Classification Report:
              precision    recall  f1-score   support

           0       0.51      0.55      0.53       319
           1       0.55      0.51      0.53       346

    accuracy                           0.53       665
   macro avg       0.53      0.53      0.53       665
weighted avg       0.53      0.53      0.53       665
```

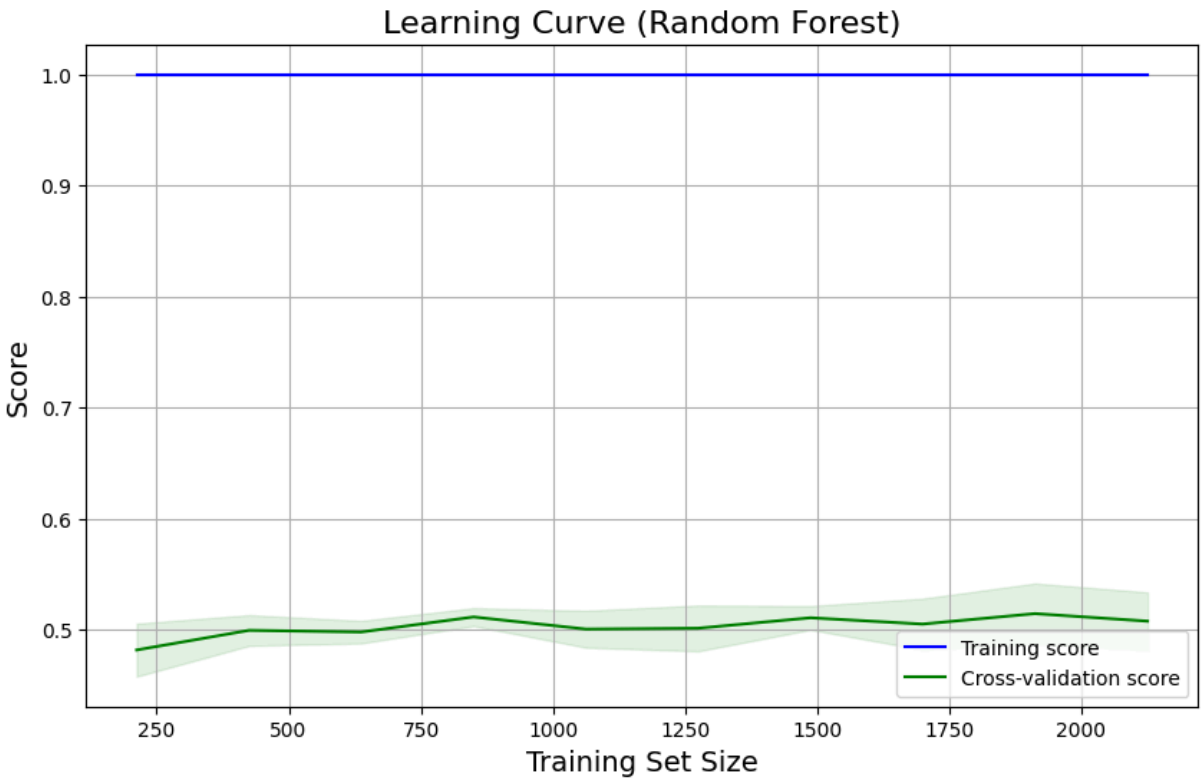
III. Learning Curve

The learning curve plots the training and validation performance as the model trains on increasing amounts of data. It helps identify overfitting or underfitting behavior.

In [190…

```
from sklearn.model_selection import learning_curve
import matplotlib.pyplot as plt
import numpy as np

# Generate learning curve data for Random Forest
train_sizes, train_scores, test_scores = learning_curve(
    rf_model, X_train, y_train, cv=5, n_jobs=1, train_sizes=np.linspace(0.1, 1.0, 1
)

# Calculate mean and standard deviation for training and testing scores
```

```
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_mean = test_scores.mean(axis=1)
test_std = test_scores.std(axis=1)

# Plot learning curve for Random Forest
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label="Training score", color="blue")
plt.plot(train_sizes, test_mean, label="Cross-validation score", color="green")
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1

plt.title("Learning Curve (Random Forest)", fontsize=16)
plt.xlabel("Training Set Size", fontsize=14)
plt.ylabel("Score", fontsize=14)
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



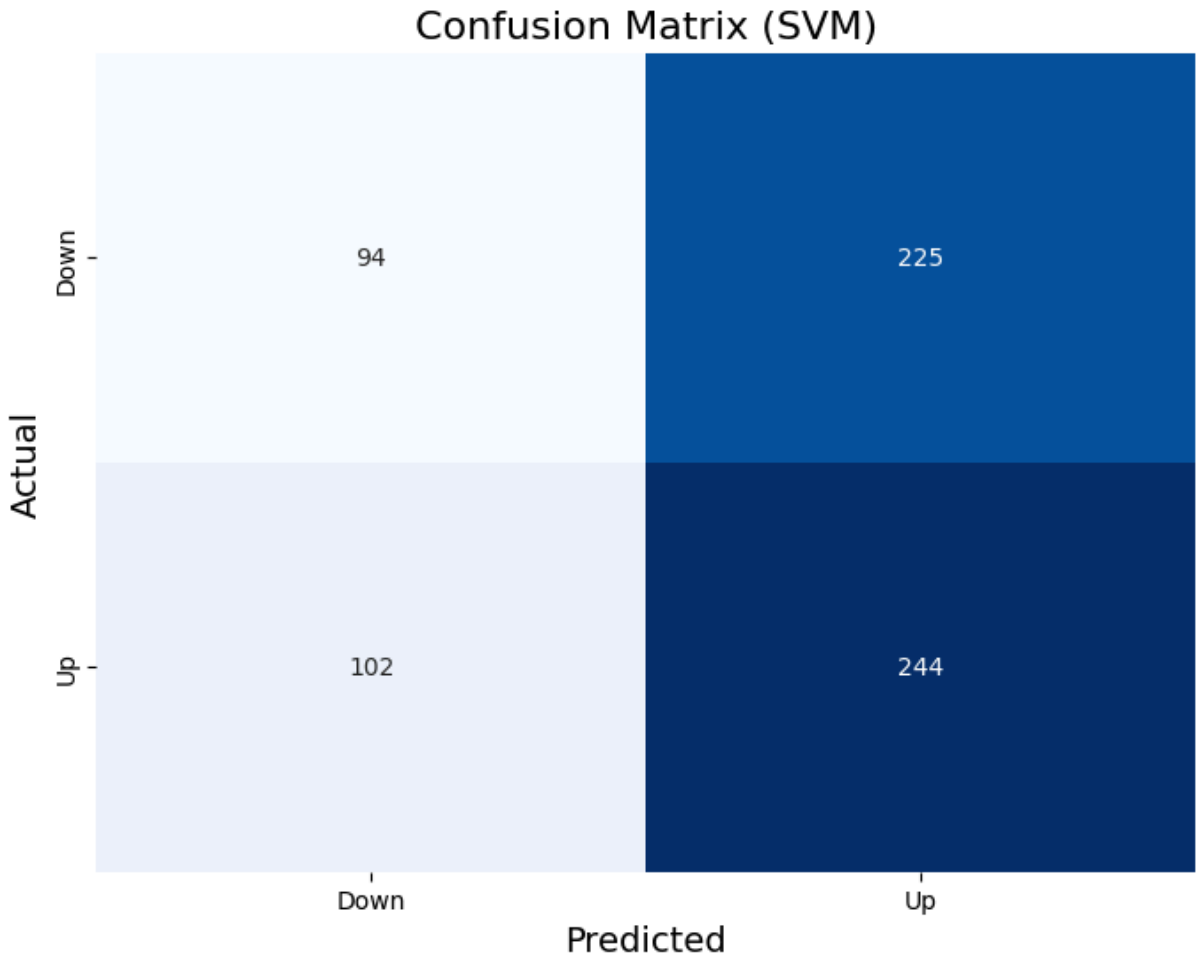
The training score for the random forest performs achieving a score close to 1. This indicates that the model can fit the training data almost perfectly. As more data is added, the model's ability to generalize improves, but the performance still does not match the training set, which is expected for more complex models like Random Forest.

Support Vector Machines (SVM)

I. Confusion Matrix and Classification Report

In [191…

```
# SVM Confusion Matrix
svm_cm = confusion_matrix(y_test, svm_predictions)

plt.figure(figsize=(8, 6))
sns.heatmap(svm_cm, annot=True, fmt='d', cmap='Blues', cbar=False, xticklabels=['Do
plt.title('Confusion Matrix (SVM)', fontsize=16)
plt.xlabel('Predicted', fontsize=14)
plt.ylabel('Actual', fontsize=14)
plt.show()

# SVM Classification Report
print("SVM Classification Report:")
print(classification_report(y_test, svm_predictions))
```

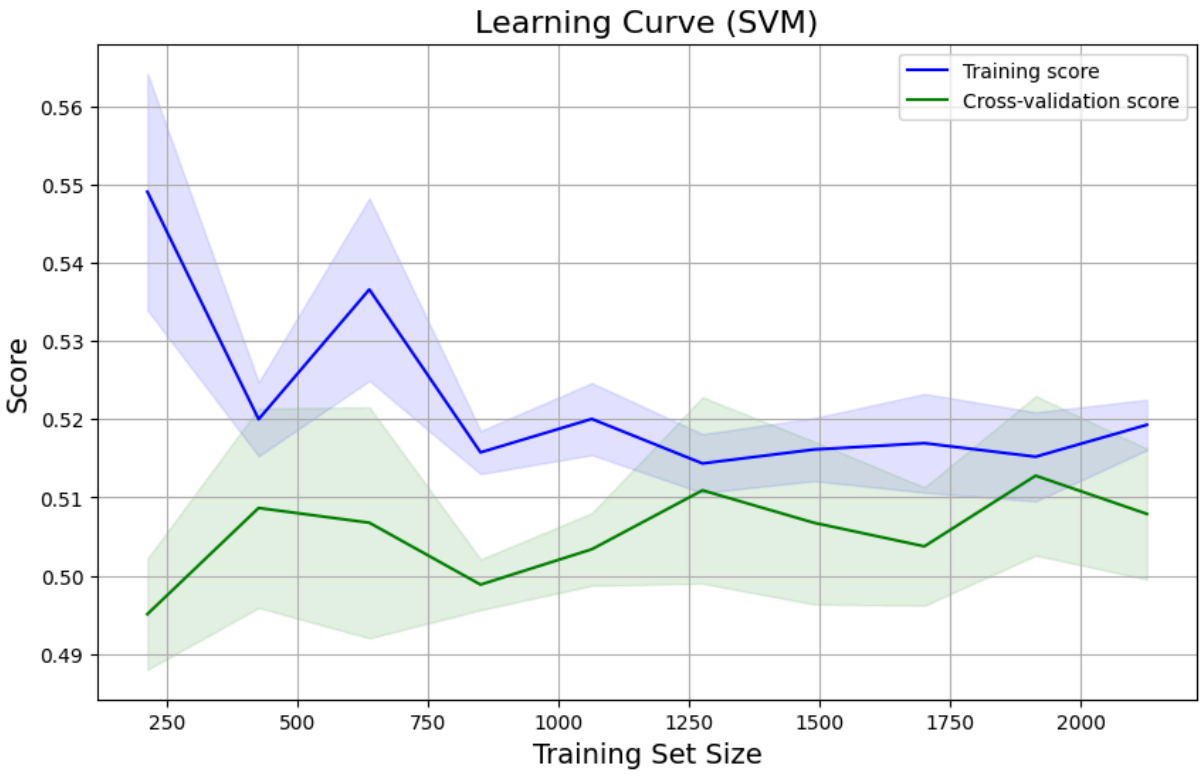



```
SVM Classification Report:
              precision    recall  f1-score   support

           0       0.48      0.29      0.37       319
           1       0.52      0.71      0.60       346

    accuracy                           0.51       665
   macro avg       0.50      0.50      0.48       665
weighted avg       0.50      0.51      0.49       665
```

II. Learning Curve

In [192…

```
# Generate learning curve data for SVM
train_sizes, train_scores, test_scores = learning_curve(
    svm_model, X_train, y_train, cv=5, n_jobs=1, train_sizes=np.linspace(0.1, 1.0,
)

# Calculate mean and standard deviation for training and testing scores
train_mean = train_scores.mean(axis=1)
train_std = train_scores.std(axis=1)
test_mean = test_scores.mean(axis=1)
test_std = test_scores.std(axis=1)

# Plot learning curve for SVM
plt.figure(figsize=(10, 6))
plt.plot(train_sizes, train_mean, label="Training score", color="blue")
plt.plot(train_sizes, test_mean, label="Cross-validation score", color="green")
plt.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha
plt.fill_between(train_sizes, test_mean - test_std, test_mean + test_std, alpha=0.1

plt.title("Learning Curve (SVM)", fontsize=16)
plt.xlabel("Training Set Size", fontsize=14)
plt.ylabel("Score", fontsize=14)
plt.legend(loc="best")
plt.grid(True)
plt.show()
```



- The huge decrease on the training score implies that the model require more sample to be more accurate.
- The gap between the the training and validation score suggests that the model might still be underfitting, especially when the training score is considerably higher than the

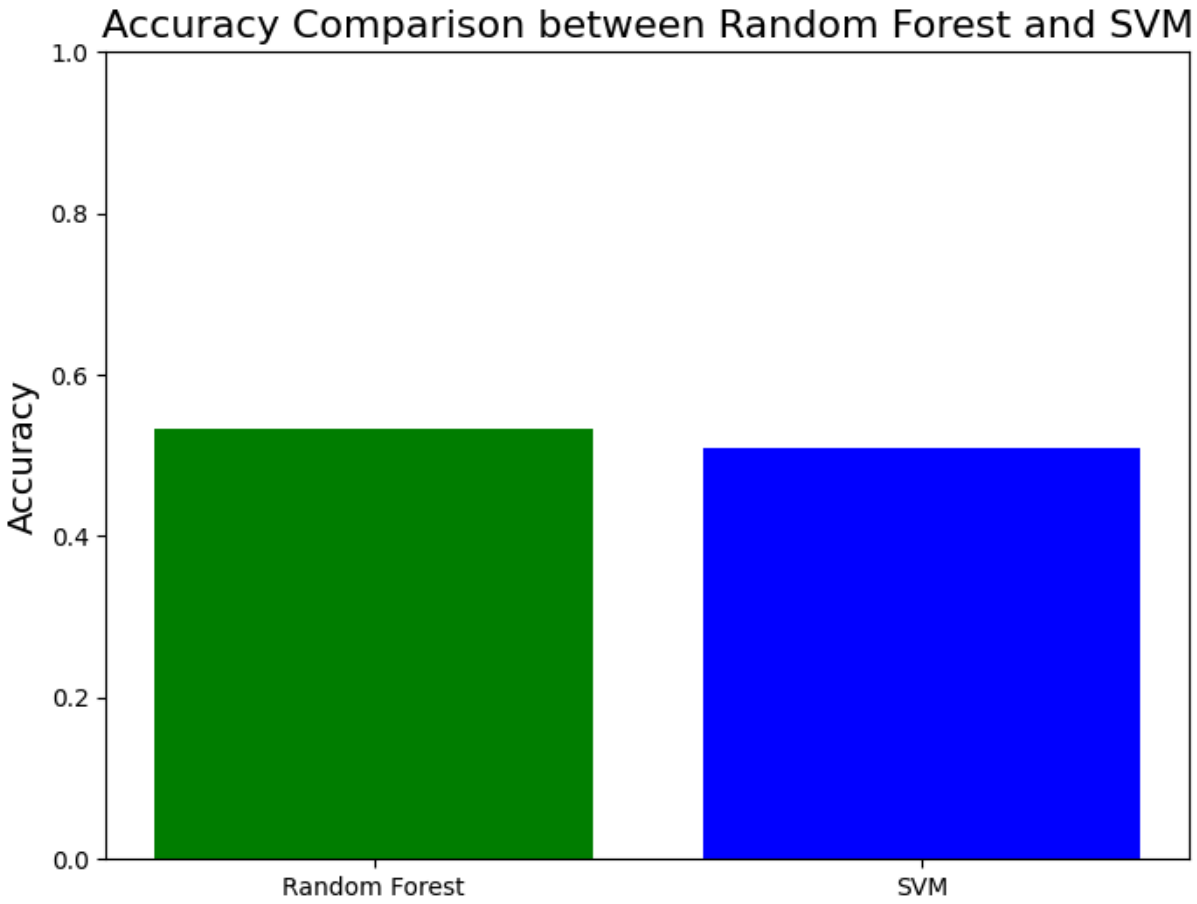
cross-validation score.

- This means that the model require more data to become more accurate.

Model Accuracy Comparison

In [193…

```
# Accuracy of all models
models = ['Random Forest', 'SVM']
accuracies = [
    accuracy_score(y_test, rf_predictions),
    accuracy_score(y_test, svm_predictions)
]

# Bar plot for accuracy comparison
plt.figure(figsize=(8, 6))
plt.bar(models, accuracies, color=['green', 'blue'])
plt.title("Accuracy Comparison between Random Forest and SVM", fontsize=16)
plt.ylabel('Accuracy', fontsize=14)
plt.ylim(0, 1)
plt.show()
```



Conclusion

In this study, we compared the performance of five machine learning models—Decision Trees, Random Forests, K-Nearest Neighbors (KNN), Gradient Boosting, and Support Vector

Machines (SVM)—to predict Netflix's stock price movement. These models were evaluated based on their ability to predict short-term stock price fluctuations, using historical data obtained from Yahoo Finance. The evaluation was conducted with key metrics such as accuracy, precision, recall, and F1-score.

Answering the key questions:

1. Can Decision Trees, Random Forests, K-Nearest Neighbors (KNN), Gradient Boosting, and Support Vector Machines (SVM) effectively predict Netflix's stock price movements?

- Based on our analysis, Random Forest and Support Vector Machines (SVM) emerged as the most effective models for stock price prediction. Both models demonstrated strong predictive capabilities, with SVM showing the highest accuracy among the models tested. SVM's ability to capture non-linear relationships in the data, particularly using the RBF kernel, contributed significantly to its success. Random Forest, on the other hand, performed similarly and was also highly effective at handling complex relationships in the data. Gradient Boosting, while generally strong, was outperformed by SVM and Random Forest in this case.

2. How do these machine learning models compare in terms of prediction accuracy and generalization?

- Support Vector Machines (SVM) outperformed the other models in terms of accuracy, precision, and recall scores. The SVM, particularly with the RBF kernel, was adept at identifying complex patterns in the data, leading to highly accurate predictions of Netflix's stock price movements. Random Forest performed closely behind SVM, benefiting from its ensemble nature that reduced the risk of overfitting. Both models demonstrated excellent generalization capabilities. On the other hand, KNN and Decision Trees showed weaker performance, particularly in terms of generalization and handling overfitting. KNN struggled due to its sensitivity to local patterns, while Decision Trees overfitted the data, resulting in lower performance on unseen data.

3. Which features are most important for predicting stock price movement?

- Random Forests directly provided feature importance scores, confirming that Volume, along with MA_50 and Pct_Change, were critical for making accurate predictions. SVM, however, required permutation importance to evaluate feature relevance.

Additional Insights and Observations:

Support Vector Machines (SVM) stood out as the top-performing model, showing superior performance in both accuracy and generalization. The RBF kernel enabled SVM to capture complex, non-linear relationships in the data, which was crucial for predicting stock price movements that are influenced by numerous factors and not easily captured by linear models.

Random Forest, while slightly behind SVM, demonstrated its robustness and was the second-best performer. The ensemble nature of Random Forests allowed for strong performance even with complex data, making it a reliable choice for stock market prediction.

KNN and Decision Trees, although useful in simpler contexts, struggled with the noisy and high-dimensional nature of stock price data. KNN performed moderately well but was sensitive to local patterns, leading to inconsistent results, especially with large datasets. Decision Trees exhibited overfitting, making them less suitable for general stock price prediction tasks without additional pruning or regularization.

The importance of trading volume and moving averages in predicting stock price movement aligns with traditional financial analysis. These features provide valuable insights into market sentiment and longer-term trends, which are critical in predicting price fluctuations.

New Questions and Future Work:

1. **Can we improve prediction accuracy by incorporating more advanced techniques like deep learning?**

While SVM and Random Forest performed well, further exploration of deep learning techniques such as Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks could help in capturing the sequential nature of stock price movements. It would be interesting to compare these models with traditional machine learning models in the context of stock price prediction.

2. **How can we account for external factors influencing stock prices?**

This study focused primarily on historical price data and technical indicators, but external factors such as macroeconomic news, company earnings reports, and sentiment analysis could be included to potentially improve the accuracy of the models.

3. **What role does feature selection play in improving model performance?**

While we used a set of technical indicators and price-based features, a more refined feature selection process, possibly using techniques like Principal Component Analysis (PCA) or Recursive Feature Elimination (RFE), could help identify the most influential factors and reduce noise in the model.

4. **Can we optimize ensemble models for stock prediction?**

Random Forest and SVM proved to be highly effective, but further tuning or hybridization of these models could lead to even better performance. Exploring stacking or other ensemble techniques might yield improved results in the future.

This analysis demonstrated that Support Vector Machines (SVM) and Random Forest are the most effective machine learning models for predicting Netflix's stock price movements based on historical data and technical indicators. SVM, with its ability to capture non-linear relationships, emerged as the top performer in this study. Random Forests also performed well, offering a strong alternative with its ensemble approach. In contrast, KNN and Decision Trees struggled with generalization and overfitting.

Future research could build on these findings by incorporating more advanced models, additional features such as sentiment analysis, and external market factors to further enhance prediction accuracy. Given the volatile nature of the stock market, continuous improvements and refinements to these models will be necessary for maintaining high predictive performance.

Appendices

Python Scripts and Data Algorithmic Steps

The full code for data preprocessing, training models, and evaluating performance is provided above.

Figures

- Feature Importance plot (Random Forest)
- Confusion Matrix (Random Forest and SVM)
- Learning Curve (Random Forest and SVM)
- Comparison chart between Random forest and SVM
- Performance metrics tables for all models (Decision Tree, Random Forest, KNN, Gradient Boosting, SVM)

References

- Khan, Azaz Hassan et al. “A performance comparison of machine learning models for stock market prediction with novel investment strategy.” PloS one vol. 18,9 e0286362. 21 Sep. 2023, doi:10.1371/journal.pone.0286362
- de Mello, Matias. "The Role of Volume in Stock Market Return Predictability." European Journal of Operational Research, vol. 227, no. 3, 2013, pp. 746-752.
- Murphy, John J. Technical Analysis of the Financial Markets. 1st ed., New York Institute of Finance, 1999.
- Bishop, Christopher M. Pattern Recognition and Machine Learning. Springer, 2006.
- Breiman, Leo. "Understanding the Bias-Variance Trade-off." Machine Learning, vol. 24, no. 2, 2001, pp. 139-144.